DATAMINERS

S.G.R.S.I

Sistema de Gestión de Recursos y Soporte de Informática

Programación Full Stack

DataMiners

Rol	Apellido	Nombre	documento identidad	email
Coordinador	Pereira	Nicolas	5.741.786-8	nicopereirasilva09@gmail.com
Sub-Coordinador	Coirolo	Bruno	5.758.905-7	coirolo794@gmail.com
Integrante 1	Camacho	Francisco	5.728.405-1	Francisco.camacho56y@gmail.com
Integrante 2	Larrosa	Alvaro	5.788.908-1	larrosaalvaro44@gmail.com

Docente: Rodriguez, Gerardo

SEGUNDA ENTREGA

24 ago 2026

CETP

DATAMINERS

Índice

1. Introducción
2. Alcance de la Segunda Entrega
3. Desarrollo del proyecto
    3.1 Frontend
    3.2 Backend
        3.2.1 Conectividad y manejo de estados HTTP3
        3.2.2 Arquitectura, paradigma POO y patrones de diseño
        3.2.3 Autenticación y autorización
    3.3 Seguridad
    3.4 Persistencia
4. Rúbrica de evaluación ponderada
5. Anexo — Sincronización con el repositorio Git
6. Referencias Bibliográficas

DATAMINERS

1. Introducción

El presente documento corresponde a la Segunda Entrega de la unidad curricular Programación Full Stack, en el marco del proyecto de egreso del Sistema de Gestión de Recursos y Soporte de Informática (SGRSI). Se detalla el desarrollo realizado a nivel de frontend y backend, la arquitectura implementada, los mecanismos de autenticación y autorización, las medidas de seguridad aplicadas y el modelo de persistencia de datos definido para el sistema.

Esta entrega constituye un avance sobre la maquetación y planificación presentadas en la primera entrega, incorporando la lógica de servidor, la conexión con la base de datos y la primera versión funcional del sistema de roles y permisos definido para los usuarios Administrador/Coordinador, Técnico/Soporte y Solicitante.

2. Alcance de la Segunda Entrega

De acuerdo a la letra del proyecto, la Segunda Entrega de Programación Full Stack debe evidenciar avances concretos en las siguientes dimensiones de trabajo:

- Frontend: adaptación de la interfaz para monitores de escritorio y pantallas de mayor resolución.
- Backend: conexión con el frontend, manejo de estados HTTP, arquitectura en tres capas y patrones de diseño.
- Autenticación y autorización: registro de usuarios, control de sesiones y protección de rutas según rol.
- Seguridad: validación y sanitización de datos en el servidor, manejo de excepciones y protección de credenciales.

- Persistencia: esquema relacional normalizado, scripts DDL y DML con datos de prueba.

La ponderación específica de cada dimensión dentro de la nota final queda a criterio del docente de la unidad curricular, en función de la metodología de trabajo implementada, la secuenciación de contenidos y el progreso del grupo a lo largo del curso.

3. Desarrollo del proyecto

3.1 Frontend

El equipo relata a continuación, con sus propias palabras, el trabajo realizado sobre la interfaz de usuario del SGRSI durante esta etapa, en línea con lo solicitado en la letra del proyecto para la Segunda Entrega.

Requerimientos cubiertos — Frontend
• Adaptación de la interfaz para monitores de escritorio (a partir de 1024 px de ancho) y resoluciones superiores.
• Implementación de pantallas funcionales de los módulos principales (inventario, mesa de ayuda, solicitudes de servicio y dashboard).
• Carga de datos de prueba en las pantallas para validar la integración visual con el backend.

3.2 Backend

3.2.1 Conectividad y manejo de estados HTTP

Requerimientos cubiertos
• Conexión estable entre el frontend y el backend del sistema.
• Manejo de estados HTTP para informar el resultado de cada operación (éxito, error de validación, no autorizado, etc.).
• Definición de las clases implementadas en el servidor y manejo de errores desde el mismo.
• Conexión con la base de datos, con justificación técnica del método de acceso a datos elegido.

3.2.2 Arquitectura, paradigma POO y patrones de diseño

Requerimientos cubiertos
• Implementación del backend siguiendo el paradigma de Programación Orientada a Objetos (POO).
• Organización del código en arquitectura de tres capas (presentación, lógica de negocio y acceso a datos).
• Utilización de patrones de diseño, con justificación lógica de su elección de acuerdo a las necesidades del sistema.

3.2.3 Autenticación y autorización

Requerimientos cubiertos
• Implementación del registro de usuarios del sistema.
• Control de sesiones, incluyendo inicio (login) y cierre (logout) de sesión.
• Protección de rutas y control de acceso según la sesión activa y el rol asignado (Administrador/Coordinador, Técnico/Soporte, Solicitante).
• Utilización de métodos de encriptación de credenciales, con justificación lógica del algoritmo elegido.

3.3 Seguridad

Requerimientos cubiertos — Seguridad
• Validación de la recepción de datos obligatoria del lado del servidor, independientemente de las validaciones ya realizadas en el cliente.
• Sanitización de entrada y salida de datos, para prevenir la inyección de código malicioso.
• Manejo estructurado de errores y excepciones en el backend.
• Configuración segura de las credenciales de acceso a la base de datos, mantenidas fuera del repositorio mediante archivo .gitignore.

3.4 Persistencia

Requerimientos cubiertos — Persistencia
• Definición de un esquema relacional con relaciones normalizadas hasta Tercera Forma Normal (3FN).
• Elaboración del DDL (Data Definition Language) para la creación de las tablas del sistema en SQL.
• Elaboración del DML (Data Manipulation Language) con datos de prueba que permiten validar el funcionamiento del sistema.

4. Rúbrica de evaluación ponderada

A continuación se presenta la rúbrica utilizada por el equipo docente para evaluar el avance del proyecto en esta entrega, correspondiente a las dimensiones trabajadas en Programación Full Stack. La ponderación de cada dimensión sobre la nota final es definida por el docente de la unidad curricular.

Dimensión	Avance destacado	Avance significativo	Avance moderado	Avance escaso / mínimo
Lógica de servidor, conectividad, autenticación y control de acceso	Clases robustas, estados HTTP precisos, conexión frontend-backend fluida, RBAC completo con login/logout y encriptación de alta seguridad.	Clases claras, estados HTTP coherentes, comunicación estable, sesiones y registro con protección de rutas por rol, encriptación justificada.	Conexión funcional con estados HTTP básicos, clases que cumplen los requisitos, login/registro funcional, rutas principales protegidas.	Estructura de servidor inicial, comunicación parcial, control de sesiones básico o autenticación sin control de roles definido.
Seguridad y validación de datos	Sanitización rigurosa de entrada/salida y manejo avanzado de excepciones. Credenciales protegidas fuera del repositorio.	Validación obligatoria en el servidor y gestión de errores mediante excepciones. Configuración segura del entorno.	Validaciones y sanitización básica en el servidor. Manejo de errores y configuración del repositorio según lo solicitado.	Validaciones elementales o parciales. Protección de credenciales sensibles mínima o inexistente.
Modelado y persistencia SQL	DER y esquema normalizado a 3FN impecables. DDL genera una estructura de base de datos optimizada.	Diagrama y esquema relacional normalizado correctamente. Las tablas creadas reflejan fielmente el modelo.	Base de datos con relaciones funcionales y normalización aceptable. El DDL permite crear las tablas necesarias.	Esquema inicial con entidades principales cubiertas o creación de tablas elemental, requiere correcciones.
Conexión backend–base de datos, arquitectura y patrones	Conexión sólida y eficiente con criterio técnico justificado. Arquitectura de tres capas con patrones de diseño avanzados.	Conexión clara y funcional, criterio técnico adecuado. Tres capas diferenciadas con patrones de diseño coherentes.	Conexión funcional con criterio reconocible. Separación de capas clara, patrones de diseño estándar.	Conexión parcial o poco consistente, o inexistente. Separación en capas identificable pero con cruces de responsabilidades.

5. Anexo — Sincronización con el repositorio Git

Conforme a lo establecido en la letra del proyecto, se actualiza el repositorio privado en Git bajo el nombre de la empresa, incorporando las correcciones indicadas por los docentes sobre la primera entrega junto con el contenido nuevo correspondiente a esta segunda entrega: backend, autenticación, seguridad y scripts de persistencia (DDL/DML).

- Convenciones de commit acordadas por el equipo.
- Historial de ramas utilizadas, con su justificación.
- Convenciones de versionado e histórico de etiquetas.

6. Referencias Bibliográficas

Las referencias se presentan conforme a las normas APA (7.ª edición), respetando los criterios de citación y formato establecidos por dicha normativa.